

SKILL_WEEK - 10 (SESSION NUMBER - 19)

Task 1: To register handlers, configure signal actions, handle interrupts, manage signal context, verify stability, and test signal events.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void handle_signal(int sig)
{
    printf("\nSignal received: %d\n", sig);
}

int main()
{
    struct sigaction sa;

    sa.sa_handler = handle_signal;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGTERM, &sa, NULL);

    printf("Process ID: %d\n", getpid());
    printf("Signal handlers registered successfully.\n");
    printf("Press Ctrl+C to generate SIGINT.\n");

    while (1)
    {
        pause();
    }

    return 0;
}
```

OUTPUT:

```
[amrutha@Amruthas-MacBook-Air OSSP % gcc src/session19_task1.c -o bin/session19_task1
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/session19_task1
Process ID: 11071
Signal handlers registered successfully.
Press Ctrl+C to generate SIGINT.
^C
Signal received: 2
```

Task 2: To capture SIGINT, forward signals, protect the shell process, terminate foreground jobs, update job states, and test interrupt handling.

```
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;

sigaction(SIGINT, &sa, NULL);

printf("Shell PID: %d\n", getpid());

pid = fork();

if (pid < 0)
{
    perror("fork failed");
    return 1;
}

if (pid == 0)
{
    setpgid(0, 0);

    signal(SIGINT, SIG_DFL);

    printf("Foreground job started.\n");
    printf("Foreground Job PID: %d\n", getpid());

    sleep(30);

    exit(0);
}

setpgid(pid, pid);
foreground_pgid = pid;

printf("Foreground Job PGID: %d\n", pid);
printf("Press Ctrl+C to interrupt foreground job.\n");

waitpid(pid, &status, 0);

if (WIFSIGNALED(status))
{
    printf("Foreground job terminated by signal %d.\n",
        WTERMSIG(status));
}

foreground_pgid = 0;

printf("Job state updated to TERMINATED.\n");
printf("Shell process is still running safely.\n");

return 0;
}
```

OUTPUT:

```
[amrutha@Amruthas-MacBook-Air OSSP % gcc src/session19_task2.c -o bin/session19_task2
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/session19_task2
Shell PID: 11512
Foreground Job PGID: 11518
Press Ctrl+C to interrupt foreground job.
Foreground job started.
Foreground Job PID: 11518
^C
SIGINT captured by shell.
Forwarding SIGINT to foreground job...
Foreground job terminated by signal 1.
Job state updated to TERMINATED.
Shell process is still running safely.
amrutha@Amruthas-MacBook-Air OSSP %
```

(SESSION NUMBER - 20)

Task 1: To capture SIGTSTP, suspend processes, update the job table, preserve state, resume later, and test suspension workflow.

```
if (job.pid == 0)
{
    printf("Child process started.\n");
    printf("Child PID: %d\n", getpid());

    printf("Sending SIGTSTP to suspend process...\n");
    raise(SIGTSTP);

    printf("Child process resumed successfully.\n");
    sleep(2);
    printf("Child process completed.\n");
    exit(0);
}
else
{
    strcpy(job.status, "RUNNING");

    waitpid(job.pid, &status, WUNTRACED);

    if (WIFSTOPPED(status))
    {
        strcpy(job.status, "STOPPED");

        printf("\nJob Table Updated:\n");
        printf("Job ID: %d\n", job.job_id);
        printf("PID: %d\n", job.pid);
        printf("Status: %s\n", job.status);

        printf("\nProcess state preserved.\n");
        printf("Resuming stopped process...\n");

        kill(job.pid, SIGCONT);

        strcpy(job.status, "RUNNING");

        printf("Status updated to %s\n", job.status);
    }

    waitpid(job.pid, &status, 0);

    strcpy(job.status, "COMPLETED");

    printf("\nFinal Job Status: %s\n", job.status);
    printf("Suspension workflow tested successfully.\n");
}

return 0;
}
```

OUTPUT:

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/session20_task1.c -o bin/session20_task1
amrutha@Amruthas-MacBook-Air OSSP % ./bin/session20_task1
Child process started.
Child PID: 12006
Sending SIGTSTP to suspend process...

Job Table Updated:
Job ID: 1
PID: 12006
Status: STOPPED

Process state preserved.
Resuming stopped process...
Status updated to RUNNING
Child process resumed successfully.
Child process completed.

Final Job Status: COMPLETED
Suspension workflow tested successfully.
amrutha@Amruthas-MacBook-Air OSSP %
```

Task 2: To create process groups, assign group IDs, transfer terminal control, restore ownership, coordinate signals, and validate behavior.

```
if (child == 0)
{
    setpgid(0, 0);

    signal(SIGTTOU, SIG_DFL);
    signal(SIGTTIN, SIG_DFL);
    signal(SIGTSTP, SIG_DFL);
    signal(SIGINT, SIG_DFL);

    printf("Child PID: %d\n", getpid());
    printf("Child Process Group ID: %d\n", getpgrp());

    sleep(2);

    printf("Foreground child process completed.\n");
    exit(0);
}
else
{
    setpgid(child, child);

    printf("Created Process Group: %d\n", child);

    if (tcsetpgrp(STDIN_FILENO, child) == 0)
        printf("Terminal control transferred to child group.\n");
    else
        perror("tcsetpgrp");

    waitpid(child, &status, 0);

    if (tcsetpgrp(STDIN_FILENO, shell_pgid) == 0)
        printf("Terminal control restored to shell.\n");
    else
        perror("tcsetpgrp restore");

    printf("Process group execution verified successfully.\n");
}

return 0;
}
```

OUTPUT:

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/session20_task2.c -o bin/session20_task2
amrutha@Amruthas-MacBook-Air OSSP % ./bin/session20_task2
Shell Process Group ID: 12451
Created Process Group: 12454
Terminal control transferred to child group.
Child PID: 12454
Child Process Group ID: 12454
Foreground child process completed.
Terminal control restored to shell.
Process group execution verified successfully.
amrutha@Amruthas-MacBook-Air OSSP %
```